

Praktikum Verteilte Systeme

1 Übersicht

Im Rahmen des Praktikums Verteilte Systeme soll eine Anwendung aus dem Bereich der *Robotik* entwickelt werden. Dazu sollen die Technologien *Sockets*, *Remote Procedure Calls (RPCs)* sowie *Message-Oriented-Middleware (MOM)* verwendet werden. Die Anwendung entsteht dabei Schritt für Schritt und jede der unten stehenden Aufgaben erweitert das System um eine Komponente oder Funktionalität.

2 Grundregeln

Folgende Regeln gelten für das gesamte Praktikum und alle Aufgaben:

- Das Bearbeiten der Praktikumsaufgaben findet soweit wie möglich in Zweier-Teams statt. Sie bekommen zu Beginn der Lehrveranstaltung einen Partner zugeteilt.
- Die Praktikumsaufgaben müssen zuhause vor- und nachbereitet werden, d.h. außerhalb der Praktikumstermine.¹
- Alle Aufgaben müssen spätestens zum letzten Gruppentermin abgenommen sein. Andernfalls gilt die "regelmäßige Teilnahme" am Praktikum als *nicht bestanden* und die ECTS-Kreditpunkte können unabhängig vom Klausurergebnis *nicht* vergeben werden. Jede Gruppe ist für die Abnahme der Aufgaben selbst verantwortlich.
- Jeder Gruppe wird ein Repository im GitLab der Hochschule (<https://gitlab.informatik.hs-fulda.de/distributed-systems-ws2425>) zur Verfügung gestellt. Dieses Repository ist für das Praktikum zu verwenden. Schreiben Sie zur Einrichtung eine Mail mit den in Gitlab registrierten Mailadressen der Gruppenmitglieder (üblicherweise vorname.nachname@informatik.hs-fulda.de) an den Dozenten.
- Die Lösungen müssen im main-Branch Ihres Repositories zur Verfügung gestellt werden. Eigene Repositories sind nicht zulässig.
- Es ist ein Build Tool (Make, cMake, Maven, Gradle, etc.) zu verwenden.
- Die Lösungen müssen containerisiert und mittels Docker und Docker-Compose zu testen sein.
- Die Aufgaben werden im Rahmen der Praktikumstermine mit dem Dozenten durchgesprochen. Hierbei müssen alle Teammitglieder anwesend sein und die Lösung erklären können.
- Es sind keine Programmiersprachen vorgegeben und eine Kombination mehrerer Sprachen ist erlaubt.
- Sockets und HTTP müssen nativ, d.h. ohne externe Bibliotheken, implementiert werden. Alle weiteren Komponenten und Bibliotheken, z.B für Thrift, gRPC, MQTT, aber auch für Logging oder

¹Ausnahme: Vor Aufgabe 0 muss nichts vorbereitet werden – außer vielleicht die Aufgabenstellung zu lesen.

die Konfiguration, dürfen frei gewählt werden. Bei Fragen besprechen Sie Ihre Pläne mit Ihrem Dozenten.

- Jede Lösung muss getestet werden. Schreiben Sie zu jeder ihrer Lösungen mindestens einen funktionalen Test sowie mindestens einen nicht funktionalen Test.

3 Aufgabenstellung

Im Rahmen des Praktikums sollen unterschiedliche Systeme rund um das Thema *Robotik* bzw. *Roboterfussball* simuliert werden. Dazu ist in mehreren Phasen jeweils ein Teil des in Abbildung 1 dargestellten Gesamtsystems zu erstellen. Am Ende müssen mehrere Roboter, die über einen Controller gesteuert werden, mittels eines verteilten Leader-Election Algorithmus einen Kapitän wählen. Das Gesamtsystem soll möglichst robust sein und Ausfälle einzelner Systeme, z.B. eines Roboters, verkraften können.

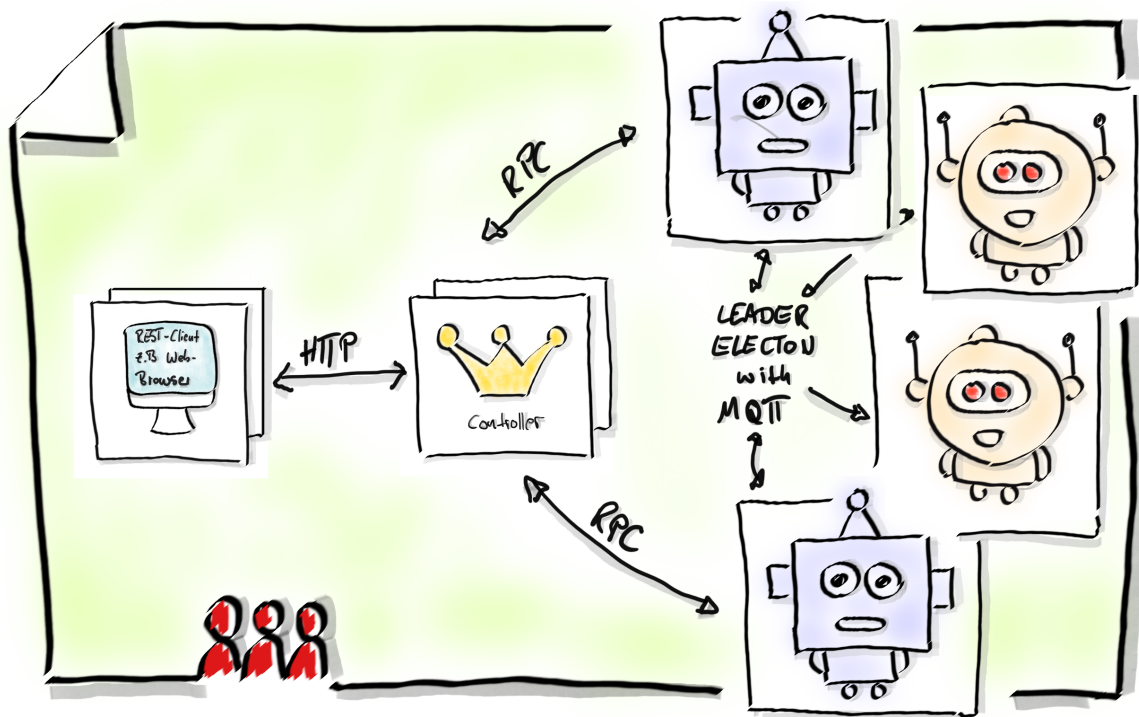


Abbildung 1: Das Gesamtsystem mit einem Client, einem hochverfügbaren Controller und mehreren Robotern.



Der Fokus liegt auf der Kommunikation der verteilten Komponenten untereinander sowie dem Software-Design, dem Testen und dem Ausrollen (Deployen) der verteilten Anwendungen.

3.1 Aufgabe 0 - Vorbereitung

Der erste Praktikumstermin dient der Vorbereitung und Planung des gesamten weiteren Praktikums. Finden Sie sich in Ihrem Team zusammen und klären Sie unbedingt gegenseitige Erwartungshaltungen. Bereiten Sie Ihr System (Entwicklungsumgebung, Docker, Kommunikationskanäle, etc.) vor. Testen Sie Ihren GitLab-Zugang und den Zugang zum zur Verfügung gestellten Repository.

In der Vergangenheit hat es sich als nützlich erwiesen, wenn die Gruppen einen Projektplan und Zeitrahmen erstellt haben. Überlegen Sie daher gemeinsam, wer wann welche Teile der Softwareentwicklung abgeschlossen haben soll. Nutzen Sie z.B. GitLab Issues oder GitLab Milestones für das Projektmanagement.

3.2 Aufgabe 1 - TCP Sockets, HTTP und REST

Im ersten Schritt soll die erste Version des Controllers implementiert werden, der eine REST-Schnittstelle über HTTP anbietet. Über die REST-Schnittstelle sollen zunächst Dummy-Daten mittels HTTP POST entgegengenommen und in einer internen Datenstruktur abgespeichert werden. Zudem bietet der Controller Schnittstellen um vorhandenen Daten mittels HTTP GET abzurufen.

Es sollen Schnittstellen für mindestens die folgenden Funktionalitäten implementiert werden:

- Abfragen des Status des Gesamtsystems (z.B. wie viele Roboter sind aktiv)
- Abfragen des aktuellen Kapitäns
- Abfragen des Zustands des Controllers (z.B. Health-Check)
- Anstoßen einer (neuen) Kapitänswahl

Der *HTTP* Server des Controllers soll nur mit Hilfe von Sockets und ohne weitere Hilfsmittel (d.h. ohne externe Bibliotheken) implementiert werden. Der Server soll *HTTP POST* und *HTTP GET* Anfragen akzeptieren. Andere Anfragen sollen abgefangen und mit einer sinnvollen Fehlerbehandlung (Fehlermeldung) behandelt werden. Es ist hierbei erforderlich, dass eingehende HTTP Anfragen komplett und korrekt eingelesen und verarbeitet werden. Das bedeutet u.a., dass GET und POST unterschieden werden müssen und, dass es nicht ausreichend ist, nur die erste Zeile eine HTTP Nachricht zu lesen.

Testen Sie zudem die Performance Ihrer Implementierung indem Sie auf der Client-Seite die Round-Trip-Time (RTT) einer HTTP POST Anfrage messen. Werten Sie den Test statistisch valide aus und erstellen Sie ein Messprotokoll, das den Test und die Ergebnisse darstellt.

3.3 Aufgabe 2 - Remote Procedure Calls (RPC)

Im nächsten Schritt soll das System um eine beliebige Anzahl an Robotern erweitert werden. Die Roboter-Software wird vom Controller mittels eines RPCs, z.B. Apache Thrift oder gRPC, gesteuert. Dazu sollen sich die Roboter beim Start zunächst beim Controller anmelden. Weiterhin braucht es einen Mechanismus, um den Zustand (z.B. Health-Check) eines jeden Roboters zu erfassen. Zudem kann der Controller die Roboter anweisen, einen neuen Kapitän zu wählen.

3.4 Aufgabe 3 - Message-oriented Middleware (MoM)

Die Wahl eines (neuen) Kapitäns sollen die Roboter untereinander mit einem verteilten Leader Election Algorithmus organisieren. Hierzu können Sie z.B. einen Ring, Bully oder Raft-Algorithms implementieren. Zum Austausch der benötigten Nachrichten soll eine Message-oriented Middleware, wie z.B. MQTT oder AMQP, zum Einsatz kommen. Definieren Sie dafür die notwendigen *Topics* sowie die Nachrichten, die für den Leader Election Algorithmus erforderlich sind. Überlegen Sie auch, wie Sie eine effiziente Fehlerbehandlung implementieren können um z.B. den Ausfall eines Roboters zu überleben. Außerdem muss sichergestellt sein, dass die weiteren Voraussetzungen zur Anwendung des Algorithmus eingehalten werden. Sobald ein (neuer) Kapitän gewählt ist, soll diese Information dem Controller mitgeteilt werden.²

Testen Sie zudem die Stabilität und Performance Ihres Algorithmus und Ihrer Implementierung auf sinnvolle Art und Weise. Erarbeiten Sie dazu den Test, werten Sie diesen statistisch valide aus und erstellen Sie ein Messprotokoll, das den Test und die Ergebnisse darstellt.

3.5 Optional: Aufgabe 4 - Hochverfügbarkeit

Im letzten Schritt soll der Controller um Funktionalitäten zur Hochverfügbarkeit erweitert werden. Stellen Sie sicher, dass Ihr System den Ausfall eines Controllers verkraften kann. Hierbei sind Sie im Systemdesign und in der Auswahl der verwendeten Technologie frei.

Testen Sie Ihr System um sicherzustellen, dass Ausfälle verkraftet werden. Beschreiben Sie Ihr finales System sowie dessen Tests und deren Ergebnisse im Detail. Begründen Sie Ihre Technologieauswahl.

²Bitte beachten Sie, dass der Controller zwar eine Wahl anstoßen kann, an der Auswahl des Leaders aber nicht aktiv beteiligt ist.